

DeepSeek V4 推理性能分析：Ascend 910C vs NVIDIA H20

摘要

本报告基于 Roofline 模型，对 DeepSeek V4 在两种硬件平台（Ascend 910C 和 NVIDIA H20）及三种服务场景（8K/4K、128K/4K、256K/4K 输入/输出长度）下的推理性能进行了全面分析。主要发现如下：

1. mHC 内核融合（默认开启）将 Prefill 时间缩短 3-4 倍，通过消除 mHC 的 HBM 流量。融合基线下：mHC 仅占 Prefill 时间的 25%-36%（未融合时为 84%）。
2. Decode 受限于 MoE 权重加载 —— MoE 路由专家权重加载在 910C 上占 Decode 的 38%-42%，在 H20 上占 14%-53%。
3. H20 的 Prefill 吞吐量是 910C 的 1.1-1.5 倍，在融合基线下差距大幅缩小（原始差距 1.5-2.6 倍）。
4. P/D 分离比例随输入长度增长 —— 1P:1D (8K), 2P:1D (128K), 3P:1D (256K) on 910C。
5. SP 结合 mHC 并行化（mHC-SP）可实现 2.1-4.2 倍 Prefill 加速，将 mHC 从总时间的 36%-84% 降低至 3%-7%。
6. 910C 的 EP 带宽优势（7.8 倍）支持高效的高 EP MoE 配置，而这在 H20 上成本过高。

1. 引言与方法论

1.1 DeepSeek V4 架构

DeepSeek V4 是一种混合专家模型（MoE），包含 43 个 Transformer 层：

- 2 个全注意力层（ratio=1）：标准 MQA，64 个 Q 头，1 个 KV 头
- 21 个 C4A 层（ratio=4）：4 倍 KV 压缩，配合 Lightning Index（topK=512）
- 20 个 C128A 层（ratio=128）：128 倍 KV 压缩，无需索引

关键架构特征：

- MQA (Multi-Query Attention) : 64 个 Q 头, 1 个 KV 头, head_dim=512
- Lightning Index: 64 个索引头, dim=128, 选择 top-512 压缩 KV 条目
- MoE: 256 个路由专家 (top-6) , 1 个共享专家, inter_dim=2048
- mHC (Hyper Connection) : 每个子层均有 FP32 前/后投影和 Sinkhorn 归一化
- KV 压缩: 分组投影按比例压缩 K 和 V 缓存

1.2 Roofline 模型

每个算子通过三个资源维度来刻画：

- Cube 时间: 矩阵乘法 FLOPs / (cube_tflops x 利用率)
- Vec 时间: 向量 FLOPs / (vec_tflops x 利用率)
- Mem 时间: HBM 字节数 / (带宽 x 利用率)

瓶颈 = $\text{argmax}(\text{cube}, \text{vec}, \text{mem})$ 。总时间 = $\max(\text{cube}, \text{vec}, \text{mem}) + \text{comm}$ 。

1.3 硬件平台

指标	Ascend 910C	NVIDIA H20	比率
Cube TFLOPS (BF16)	376	148	910C 2.54x
Vec TFLOPS (FP32)	24	44	H20 1.83x
Cube:Vec 比率	15.7:1	3.4:1	H20 更均衡
HBM 带宽 (GB/s)	1,800	4,000	H20 2.22x
HBM 容量 (GB)	64	96	H20 1.5x
TP 带宽 (GB/s)	392	450	接近
EP 带宽 (GB/s)	392	50	910C 7.84x
算力利用率	50%	50%	相同
HBM 带宽利用率	80%	80%	相同

最显著的硬件差异：910C 拥有 7.8 倍的 EP 带宽（节点内互联），但 HBM 带宽低 2.2 倍。这从根本上决定了最优部署策略。

1.4 服务场景

场景	输入长度	输出长度	应用场景
8K/4K	8,192	4,096	对话、编程辅助
128K/4K	131,072	4,096	长上下文 RAG、文档问答
256K/4K	262,144	4,096	全窗口文档分析

1.5 搜索约束

- Prefill NPU：8、16、32、64、128、256 个 GPU（ $\text{physical_gpus} = \text{TP} \times \text{DP}$ ）
- Decode NPU：仅 16 或 32 个 GPU（实际部署约束）
- 内存：权重 + KV 缓存必须容纳在 HBM 中（910C 为 64 GB，H20 为 96 GB）
- 并行度：TP 整除 Q 头数（64），EP 整除专家数（256）， $(\text{TP} \times \text{DP}) \% \text{EP} == 0$

2. 参数搜索结果 —— 8K 输入 / 4K 输出

2.1 Ascend 910C

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	330 ms
Prefill 吞吐	8	16	2	256	16	1,656 tps/gpu
Decode 延迟	4	32	8	8	32	19.3 ms/step
Decode 吞吐	4	16	4	512	16	181 tps/gpu

主要观察：

- Prefill 延迟在 64 个 GPU 配置下最小，使用 EP=64 以最大化专家并行度
- Prefill 吞吐在 16 个 GPU 时达到峰值（TP=8, EP=16, DP=2）—— 更大的集群并不能提升每 GPU 效率
- Decode 倾向于 TP=4 而非 TP=8，因为 Decode 受限于内存，TP=4 已提供足够的权重切分
- Decode 吞吐受益于大 Batch（BS=512），以摊薄权重加载开销

2.2 NVIDIA H20

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	554 ms
Prefill 吞吐	8	8	1	128	8	1,848 tps/gpu
Decode 延迟	4	32	8	8	32	9.0 ms/step
Decode 吞吐	8	16	2	512	16	252 tps/gpu

主要观察：

- H20 Prefill 吞吐是 910C 的 1.12 倍（1,848 vs 1,656 tps/gpu）—— 在融合基线下差距大幅缩小
- H20 仅用 8 个 GPU（EP=8）即达到最佳 Prefill 吞吐，而 910C 需要 16 个 GPU —— H20 更高的 HBM 带宽弥补了较少的专家并行副本
- H20 Decode 延迟优 2.1 倍（9.0 vs 19.3 ms）—— 主要得益于更快的权重加载
- H20 吞吐场景选择 EP=8（而非 EP=16/64），因为 EP 带宽仅 50 GB/s —— 高 EP 配置会带来巨大的 AllToAll 开销

2.3 8K/4K 硬件对比

指标	910C	H20	H20/910C
Prefill 延迟 (ms)	330	554	1.68x (910C 更快)
Prefill 吞吐 (tps/gpu)	1,656	1,848	1.12x (H20 更高)
Decode 延迟 (ms/step)	19.3	9.0	0.47x (H20 更快)
Decode 吞吐 (tps/gpu)	181	252	1.39x (H20 更高)

3. 参数搜索结果 —— 128K 及 256K 输入

3.1 128K 输入 / 4K 输出

Ascend 910C

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	10,747 ms
Prefill 吞吐	8	16	2	16	16	760 tps/gpu
Decode 延迟	4	32	8	8	32	21.0 ms/step
Decode 吞吐	4	32	8	128	32	16.7 tps/gpu

NVIDIA H20

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	20,396 ms
Prefill 吞吐	8	8	1	8	8	798 tps/gpu
Decode 延迟	4	32	8	8	32	9.9 ms/step
Decode 吞吐	4	16	4	64	16	47.4 tps/gpu

3.2 256K 输入 / 4K 输出

Ascend 910C

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	33,923 ms
Prefill 吞吐	8	16	2	8	16	482 tps/gpu
Decode 延迟	4	32	8	8	32	21.6 ms/step
Decode 吞吐	4	32	8	64	32	8.5 tps/gpu

场景	TP	EP	DP	BS	GPU 数	指标
Prefill 延迟	8	64	8	8	64	65,685 ms
Prefill 吞吐	8	8	1	4	8	497 tps/gpu
Decode 延迟	4	32	8	8	32	10.1 ms/step
Decode 吞吐	4	32	8	128	32	25.6 tps/gpu

3.3 长上下文扩展分析

长上下文下内存约束严峻：

- 128K 时，每个 rank 的最大 Batch 降至个位数（910C 上 DP=2 时总 BS=16）
- 256K 时，最大 Batch 再减半（910C 上总 BS=8，H20 在 EP=8 时总 BS=4）
- Decode 吞吐大幅下降：910C 上 181 -> 16.7 -> 8.5 tps/gpu（8K -> 128K -> 256K）

注意力计算量以二次方增长：

- Prefill 延迟：910C 上 0.33s -> 10.7s -> 33.9s（8K -> 128K -> 256K）
- 8K 到 128K 增长约 32 倍（序列长度增长 16 倍即 256 倍，但压缩机制和融合有所缓解）
- 128K 到 256K 增长约 3.2 倍（更接近理论上的 4 倍二次方增长）

Decode 延迟相对稳定：

- 910C 在三种场景下为 19.3 -> 21.0 -> 21.6 ms/step
- Decode 每步处理 1 个 token，因此更长的上下文主要增加 KV 缓存读取开销
- KV 压缩（C4A/C128A）有效限制了 Decode 注意力开销

4. P/D 分离式服务分析

4.1 方法论

在 Prefill/Decode 分离式服务中：

- P 实例 处理输入 token（Prefill 阶段）

- D 实例 生成输出 token（Decode 阶段）
- QPS 平衡： $N_p \times P_rps \geq N_d \times D_rps$

其中：

```
P_rps = P_tps_per_instance / input_len
D_rps = D_tps_per_instance / output_len
N_p / N_d >= (D_tps_instance x input_len) / (P_tps_instance x output_len)
```

4.2 结果

Ascend 910C

场景	P 配置	P tps/实例	D 配置	D tps/实例	P:D 比例	总 GPU 数
8K/4K	TP=8,EP=16,DP=2 (16 GPUs)	26,490	TP=4,EP=16,DP=4 (16 GPUs)	2,897	1P:1D	32
128K/4K	TP=8,EP=16,DP=2 (16 GPUs)	12,155	TP=4,EP=32,DP=8 (32 GPUs)	533	2P:1D	64
256K/4K	TP=8,EP=16,DP=2 (16 GPUs)	7,707	TP=4,EP=32,DP=8 (32 GPUs)	273	3P:1D	80

NVIDIA H20

场景	P 配置	P tps/实例	D 配置	D tps/实例	P:D 比例	总 GPU 数
8K/4K	TP=8,EP=8,DP=1 (8 GPUs)	14,786	TP=8,EP=16,DP=2 (16 GPUs)	4,026	1P:1D	24
128K/4K	TP=8,EP=8,DP=1 (8 GPUs)	6,382	TP=4,EP=16,DP=4 (16 GPUs)	759	4P:1D	48
256K/4K	TP=8,EP=8,DP=1 (8 GPUs)	3,973	TP=4,EP=32,DP=8 (32 GPUs)	820	14P:1D	144

4.3 分析

P:D 比例由输入/输出长度不平衡驱动：

- 8K/4K（比例 2:1）时，P:D 约 1:1 —— Prefill 和 Decode 按比例扩展
- 128K/4K（比例 32:1）时，P:D 约 2-4:1 —— Prefill 成为瓶颈
- 256K/4K（比例 64:1）时，P:D 约 3-14:1 —— 更加偏重 Prefill

910C 在长上下文时需要的 P 实例数少于 H20：

- 128K：910C 需要 2P:1D（64 GPUs），而 H20 需要 4P:1D（48 GPUs）
- 256K：910C 需要 3P:1D（80 GPUs），而 H20 需要 14P:1D（144 GPUs）
- 910C 的 mHC 融合大幅提升了 Prefill 吞吐，使 P:D 比例更有利

GPU 总量随上下文长度显著增长：

- 8K/4K：32 个 GPU（910C），24 个 GPU（H20）
- 128K/4K：64 个 GPU（910C），48 个 GPU（H20）
- 256K/4K：80 个 GPU（910C），144 个 GPU（H20）

在 256K 场景下，H20 需要比 910C 更多的 GPU，因为 H20 的 Decode 吞吐较高导致需要更多 P 实例来匹配。

5. SP 及 mHC-SP 分析

5.1 三种测试配置

使用 TP=8、EP=16、DP=2 及代表性 Batch 大小进行测试：

1. 无 SP（`sp=False, mhc_sp=False`）—— 基线配置
2. 仅 SP（`sp=True, mhc_sp=False`）—— 对 RMSNorm/激活进行序列并行；mHC 仍在 T_full 下运行
3. SP + mHC-SP（`sp=True, mhc_sp=True`）—— mHC 操作也沿 TP 维度并行化

5.2 Prefill 时间对比 (ms)

Ascend 910C

场景	无 SP	仅 SP	SP + mHC-SP	加速比 (无 SP -> SP+mHC-SP)
8K/4K	3,507	2,492	1,706	2.06x
128K/4K	102,647	86,264	73,696	1.39x
256K/4K	152,440	136,066	123,497	1.23x

NVIDIA H20

场景	无 SP	仅 SP	SP + mHC-SP	加速比 (无 SP -> SP+mHC-SP)
8K/4K	15,181	7,842	4,475	3.39x
128K/4K	336,214	218,712	164,846	2.04x
256K/4K	871,651	636,641	528,911	1.65x

5.3 关键发现

SP 单独使用在 910C 上可获得 29% 加速（8K），但在 H20 上收益更显著（48%）：

- 在 910C 上，SP 减少了 Norm 和 MoE 路由时间，mHC 融合基线下 SP 效果更明显
- 在 H20 上，SP 还额外降低了更大的通信开销（由于低 EP 带宽，H20 上 EP AllToAll 开销巨大）

mHC-SP 在融合基线下仍有价值：

- 8K 时：mHC 从融合基线的 36% 降至 6.6%（mHC 绝对时间减少约 8 倍）
- 128K 时：mHC 从 16.7% 降至 2.4%
- 256K 时：mHC 从 10.6% 降至约 1.5%
- 8 倍 mHC 缩减来自 TP=8 对内存受限的 FP32 操作的并行化

mHC-SP 的收益随序列长度增加而递减：

- 8K 时，mHC 在融合基线下仍是重要瓶颈 -> mHC-SP 带来 2.1 倍加速
- 256K 时，注意力计算（二次方增长）开始占主导 -> mHC-SP 带来 1.2 倍加速
- 这表明对于长上下文工作负载，注意力优化比 mHC 优化更为重要

5.4 类别分布变化（910C，8K/4K）

类别	无 SP	仅 SP	SP + mHC-SP
mHC	25.6%	36.0%	6.6%
通信	31.9%	18.6%	27.1%
注意力投影	17.9%	25.2%	36.9%
注意力计算	5.5%	7.7%	11.3%
MoE（全部）	10.8%	3.0%	4.4%
其他	8.4%	9.5%	13.9%

应用 mHC-SP 后，瓶颈转移至注意力投影和通信 —— 表明进一步优化应聚焦这些领域。

6. mHC 内核融合优化分析

6.1 优化级别

四个逐步激进的 mHC 优化级别：

级别	mhc_kernel_fused	mhc_sp	mhc_fused_bf16	描述
未融合 FP32	False	False	False	原始基线
融合 FP32	True	False	False	内核融合（新默认值）
融合 FP32 + SP	True	True	False	+ mHC 序列并行
融合 BF16 + SP	True	True	True	+ BF16 精度

6.2 Prefill 时间对比（910C，8K/4K，TP=8, EP=16, DP=2）

优化级别	Prefill 时间 (ms)	相对未融合基线	mHC 占比
未融合 FP32	10,144	1.00x	84.3%
融合 FP32（新默认）	2,492	4.07x 加速	36.0%
融合 FP32 + SP	1,706	5.94x 加速	6.6%

优化级别	Prefill 时间 (ms)	相对未融合基线	mHC 占比
融合 BF16 + SP	1,650	6.15x 加速	3.4%

6.3 瓶颈迁移

融合前后的类别分布变化（910C，8K/4K）：

类别	未融合 FP32	融合 FP32	融合 FP32 + SP	融合 BF16 + SP
mHC	84.3%	36.0%	6.6%	3.4%
注意力投影	6.2%	25.2%	36.9%	38.1%
注意力计算	1.9%	7.7%	11.3%	11.7%
通信	4.6%	18.6%	27.1%	28.0%
Lightning Index	1.2%	4.9%	7.1%	7.4%
MoE	0.7%	3.0%	4.4%	4.5%
其他	1.1%	4.6%	6.7%	6.9%

融合后瓶颈从 mHC（MEM 受限）迁移至注意力投影（CUBE 受限）——这正是 910C 的强项（2.54 倍 Cube TFLOPS 优势）。

6.4 代表层算子详情（910C，8K/4K，C4A 层）

关键 mHC 算子时间变化：

算子	未融合 FP32	融合 FP32	融合 FP32 + SP	融合 BF16 + SP
mhc_pre_attn	80.5 ms	6.7 ms	0.84 ms	0.42 ms
mhc_post_attn	18.6 ms	—	—	—
mhc_post_attn_pre_moe	—	7.5 ms	0.93 ms	0.47 ms
mhc_pre_moe	80.5 ms	—	—	—
mhc_post_moe	18.6 ms	6.7 ms	0.84 ms	0.42 ms
sinkhorn_attn	0.24 ms	0.24 ms	0.24 ms	0.24 ms
sinkhorn_moe	0.24 ms	0.24 ms	0.24 ms	0.24 ms

- 融合将 mhc_pre 从 80.5ms 降至 6.7ms（12 倍），通过消除中间 HBM 读写

- SP 在 TP=8 下进一步将融合 mHC 分摊 8 倍
- BF16 将 FP32 数据量减半，再获约 2 倍提升

6.5 关键洞察

1. 内核融合将 mHC 的 HBM 流量减少约 10 倍 —— 中间结果保留在寄存器/SRAM 中而非写回 HBM
2. 融合默认配置使 910C 在 Prefill 上与 H20 具有竞争力 —— 8K/4K Prefill 吞吐差距从 2.57 倍缩小至 1.12 倍
3. BF16 融合提供额外约 30% 的性能提升 —— 从 1,706ms 降至 1,650ms (8K/4K)
4. 融合后瓶颈迁移至注意力投影 (CUBE 受限) —— 910C 的 2.54 倍 Cube 优势可在此充分发挥

7. 算子瓶颈分析

7.1 各类别瓶颈总结

类别	Prefill 瓶颈	Decode 瓶颈
注意力投影	CUBE	MEM
注意力计算	CUBE	MEM
KV 压缩	CUBE	VEC/MEM
Lightning Index	COMM (得分 AllReduce)	COMM
mHC 前/后投影	MEM (FP32)	MEM (FP32)
Sinkhorn	VEC (FP32)	MEM (FP32)
MoE 门控	CUBE	MEM
MoE 路由专家	CUBE	MEM
通信	COMM	COMM

关键洞察：Prefill 受限于计算 (CUBE/VEC)，Decode 受限于内存 (MEM/COMM)。

这是经典的 LLM 推理模式 —— Prefill 处理大量 token (计算密集型矩阵乘法)，而 Decode 每步生成一个 token (权重加载主导)。

7.2 mHC：异常组件

mHC 操作的独特之处在于全程使用 FP32（每元素 4 字节，而 BF16 为 2 字节）。这使内存流量翻倍并改变了计算-内存比：

- mHC 前/后投影：即使在 Prefill 阶段也受限于内存（FP32 使内存翻倍，而 Cube FLOPs 未相应增长）
- Sinkhorn：纯 VEC 操作（无矩阵乘法），在 910C 的 Prefill 阶段受限于 VEC
- 在 H20 上，由于 Vec TFLOPS 高 1.83 倍，Sinkhorn 的相对开销较小
- 内核融合（默认开启）将 mHC 的 HBM 流量减少约 10 倍，使 mHC 从 84% 降至 36%

7.3 Prefill 算子分布（910C，最优吞吐配置）

类别	8K/4K	128K/4K	256K/4K
mHC	36.3%	16.7%	10.6%
注意力计算	7.8%	31.6%	39.0%
Lightning Index	4.8%	28.2%	35.5%
注意力投影	25.4%	11.7%	7.4%
通信	18.1%	8.3%	5.3%
MoE	3.0%	1.4%	0.9%
其他	4.6%	2.1%	1.3%

随着序列长度增加，注意力计算以二次方增长，削弱了 mHC 的主导地位。在 256K 时，注意力 + 索引合计占 74.5%。

7.4 Decode 算子分布（910C，最优吞吐配置）

类别	8K/4K	128K/4K	256K/4K
MoE 路由	55.9%	40.3%	41.1%
mHC	4.1%	0.7%	0.4%
注意力计算	1.0%	22.0%	21.4%
通信	12.9%	12.4%	12.2%
Lightning Index	8.2%	16.4%	16.7%

类别	8K/4K	128K/4K	256K/4K
注意力投影	5.4%	6.8%	6.9%
其他	12.5%	1.4%	1.3%

Decode 以 MoE 权重加载为主导（40%–56%）。随着序列长度增加，注意力和 Lightning Index KV 缓存读取占比增长。

8. 各类别时间分布 —— 详细表格

8.1 Prefill（H20，最优吞吐配置）

类别	8K/4K	128K/4K	256K/4K
注意力投影	36.0%	11.7%	7.4%
mHC	9.1%	5.9%	3.9%
注意力计算	11.0%	31.6%	39.0%
Lightning Index	4.4%	28.2%	35.5%
通信	26.6%	15.1%	10.5%
MoE	6.6%	2.6%	1.6%
KV 压缩	3.4%	1.1%	0.7%
其他	2.8%	3.8%	1.4%

H20 的 mHC 占比低于 910C，因为 H20 更高的 HBM 带宽（4000 vs 1800 GB/s）进一步缓解了 FP32 内存瓶颈。

8.2 Decode（H20，最优吞吐配置）

类别	8K/4K	128K/4K	256K/4K
注意力投影	10.0%	4.2%	4.5%
mHC	2.3%	0.4%	0.2%
注意力计算	8.9%	13.5%	13.9%

类别	8K/4K	128K/4K	256K/4K
MoE 路由	46.7%	49.5%	53.4%
通信	18.2%	16.6%	13.2%
Lightning Index	10.1%	11.0%	11.9%
其他	3.8%	4.8%	2.9%

在 H20 上，Decode 中 MoE 权重加载始终占据主导（47%–53%），原因在于 H20 的大 HBM 带宽使权重加载相对更快但仍是最大开销。

9. Ascend 910C vs NVIDIA H20 —— 详细对比

9.1 Prefill 吞吐对比

场景	910C (tps/gpu)	H20 (tps/gpu)	H20/910C	910C GPU 数	H20 GPU 数
8K/4K	1,656	1,848	1.12x	16	8
128K/4K	760	798	1.05x	16	8
256K/4K	482	497	1.03x	16	8

mHC 融合后 H20 优势大幅缩小：1. 融合消除了 mHC 的 HBM 瓶颈，910C 的劣势减弱 2. 注意力计算（CUBE 受限）增长，而 910C 拥有 2.54 倍的 Cube TFLOPS 3. 长上下文时差距更小，因为注意力（CUBE）占比更大

9.2 Decode 吞吐对比

场景	910C (tps/gpu)	H20 (tps/gpu)	H20/910C	910C GPU 数	H20 GPU 数
8K/4K	181	252	1.39x	16	16
128K/4K	16.7	47.4	2.84x	32	16
256K/4K	8.5	25.6	3.01x	32	32

H20 在长序列 Decode 中优势增大，原因是：1. 更长的上下文 -> 更大的 KV 缓存 -> 更多 HBM 读取 -> H20 的 2.2 倍带宽优势更加重要 2. 910C 需要 32 个 GPU（更大 EP），而 H20 在短序列时仅需 16 个 GPU

9.3 Decode 延迟对比

场景	910C (ms/step)	H20 (ms/step)	H20/910C
8K/4K	19.3	9.0	0.47x (H20 更快)
128K/4K	21.0	9.9	0.47x
256K/4K	21.6	10.1	0.47x

H20 Decode 延迟在所有上下文长度下均保持约 2 倍优势，直接反映了 HBM 带宽比率。

9.4 根因分析

910C 的优势场景：

- Cube 密集型工作负载：注意力投影（矩阵乘法主导）、MoE 专家矩阵乘法
- 高 EP 配置：EP=32/64 成本较低，得益于 392 GB/s 的 EP 带宽
- mHC 融合后 Prefill 延迟：910C 在 8K/4K 下 330ms vs H20 的 554ms（910C 更快 1.68 倍）

H20 的优势场景：

- 内存受限工作负载：Decode 权重加载、KV 缓存读取
- 低 EP 吞吐：EP=8 仅用 8 个 GPU 即可匹配或超过 910C 的 16 GPU 配置
- 所有场景下的 Decode 阶段（始终快 2 倍）

H20 的 EP 带宽问题：

- H20 EP 带宽仅 50 GB/s（IB 跨节点），而 910C 为 392 GB/s（片上互联）
- EP=64 时，H20 的 AllToAll 时间约为 910C 的 7.8 倍
- 这迫使 H20 在吞吐配置中使用 EP=8，即 256 个专家仅分布在 8 个 rank 上（每 rank 32 个专家）
- EP=8 时每 rank 权重内存更高，但 H20 的 96 GB HBM 可容纳

10. 行业启示

10.1 KV 缓存与长上下文

长上下文（128K+）从根本上改变了服务格局：

- 由于 KV 缓存内存限制，每 rank Batch 大小缩减至 1-8
- 从 8K 到 128K/256K，Decode 吞吐下降 10-21 倍
- P/D 比例增加 2-14 倍，需要相应更多的 Prefill 实例

启示：KV 压缩（如 DeepSeek V4 的 C4A/C128A）对于实际的长上下文服务至关重要。若没有压缩，128K 上下文在合理 Batch 大小下将不可行。

10.2 P/D 分离

分析证实 P/D 分离对长上下文服务至关重要：

- 128K/4K 时：910C 需要 2P:1D，H20 需要 4P:1D
- 256K/4K 时：910C 需要 3P:1D，H20 需要 14P:1D
- 混合（非分离）服务会在非瓶颈阶段浪费 GPU 资源

实践指导：对于 910C 上的 128K 工作负载，64 个 GPU 的集群（2x16 GPU Prefill + 1x32 GPU Decode）可提供 QPS 均衡的服务。

10.3 硬件设计启示

Cube:Vec 比率对 mHC 工作负载的影响：

- 910C（15.7:1）在 mHC Sinkhorn（纯 VEC）运算期间，Vec/Mem 大量闲置
- H20（3.4:1）在混合 CUBE/VEC 工作负载上更为均衡
- 未来 NPU 应考虑为 FP32 归一化层提供更高的 Vec TFLOPS

HBM 带宽是 Decode 的决定性因素：

- H20 的 2.2 倍带宽优势直接转化为一致的 2 倍 Decode 加速
- 对于 Decode 密集型工作负载（如长输出的聊天机器人），HBM 带宽 > 计算 TFLOPS

HBM 容量支持更大 Batch:

- H20 的 96 GB 相比 910C 的 64 GB，可容纳多 50% 的配置
- 在 128K/256K 时，H20 可适配 910C 无法容纳的 Batch 大小

10.4 MoE 网络需求

EP 带宽是 MoE 模型服务的关键差异因素:

- 910C 的 392 GB/s EP 带宽允许 EP=64 配置仅带来适度的 AllToAll 开销
- H20 的 50 GB/s EP 带宽使 EP=64 配置代价极高 (AllToAll 开销高 7.8 倍)
- H20 通过使用较低 EP (EP=8) 并在每 rank 放置更多专家来弥补

启示: MoE 模型强烈偏好高带宽互联平台。NVLink 域 (8 GPU) 在 H20 上高效运行, 但跨节点 MoE 开销很大。

10.5 mHC 作为新范式

DeepSeek V4 的 mHC (Hyper Connection) 在架构上具有创新性, 但计算开销较大:

- 未融合时占 Prefill 时间的 53%–84%
- 使用 FP32 运算, 内存流量为 BF16 的两倍
- Sinkhorn 归一化是纯 VEC 操作, 无矩阵乘法并行性

内核融合量化 (默认开启):

- 融合将 mHC HBM 流量减少约 10 倍 (中间结果保留在寄存器/SRAM)
- mHC 占比从 84% 降至 36% (融合 FP32), 再到 3.4% (融合 BF16 + SP)
- 910C 的 Prefill 吞吐差距从 2.57 倍缩小至 1.12 倍 (vs H20)

进一步优化机会: 1. mHC-SP (沿 TP 维度并行化 mHC): 在融合基线上额外 1.5–2 倍加速 2. FP16/BF16 mHC: 在 mHC-SP 基础上额外约 3% 提升 3. 融合 Sinkhorn 算子: Sinkhorn 在融合后占比不大, 优先级较低

11. 部署建议

11.1 短上下文（8K/4K）—— 对话/编程

平台	Prefill	Decode	P:D	总 GPU 数
910C	TP=8, EP=16, DP=2 (16 GPUs)	TP=4, EP=16, DP=4 (16 GPUs)	1:1	32
H20	TP=8, EP=8, DP=1 (8 GPUs)	TP=8, EP=16, DP=2 (16 GPUs)	1:1	24

- 两个平台均能高效处理 8K 场景
- H20 成本效益略优（24 vs 32 个 GPU 即可实现均衡服务）
- 910C 在 Prefill 延迟上更优（330ms vs 554ms）
- 建议在两个平台上均启用共享专家重叠

11.2 长上下文（128K/4K）—— RAG/文档问答

平台	Prefill	Decode	P:D	总 GPU 数
910C	TP=8, EP=16, DP=2 (16 GPUs)	TP=4, EP=32, DP=8 (32 GPUs)	2:1	64
H20	TP=8, EP=8, DP=1 (8 GPUs)	TP=4, EP=16, DP=4 (16 GPUs)	4:1	48

- 910C 和 H20 所需 GPU 数接近（64 vs 48）
- 910C 的 mHC 融合使 P:D 比例更有利（2:1 vs 4:1）
- 内存是 Batch 大小的首要约束

11.3 超长上下文（256K/4K）—— 全文档分析

平台	Prefill	Decode	P:D	总 GPU 数
910C	TP=8, EP=16, DP=2 (16 GPUs)	TP=4, EP=32, DP=8 (32 GPUs)	3:1	80
H20	TP=8, EP=8, DP=1 (8 GPUs)	TP=4, EP=32, DP=8 (32 GPUs)	14:1	144

- 256K 下 910C 需要的 GPU 数远少于 H20（80 vs 144）
- H20 的高 Decode 吞吐导致需要更多 P 实例来匹配
- 需要权衡 256K 上下文的 GPU 投入与分块策略的取舍

11.4 通用指导

1. 始终启用 SP (`sp=True`) —— 可免费获得 Norm/激活计算的性能提升
2. mHC 内核融合 (`mhc_kernel_fused=True`，默认开启) —— 将 mHC HBM 流量减少约 10 倍
3. 共享专家重叠 (`shared_expert_overlapped=True`，默认开启) —— 共享专家计算与 MoE 调度/汇总通信重叠
4. 探索 mHC-SP —— 若框架支持 mHC 并行化，可获得额外 1.5–2 倍 Prefill 加速
5. 根据网络调整 EP —— 910C 使用高 EP (EP=16–64)，H20 使用低 EP (EP=8–16)
6. Decode 尽量使用大 Batch —— Decode 吞吐随 Batch 大小近乎线性增长，直至内存上限
7. 128K+ 场景必须使用 P/D 分离 —— 混合服务浪费 60%–80% 的资源

附录 A：方法论细节

- Roofline 模型：每个算子时间 = $\max(\text{cube_time}, \text{vec_time}, \text{mem_time}) + \text{comm_time}$
- FLOPs：矩阵乘法 $[M,K] \times [K,N] = M \times N \times K \times 2$
- BF16：每元素 2 字节；FP32 (mHC)：每元素 4 字节
- 通信：AllReduce = $2(n-1)/n \times \text{vol}/\text{BW}$ ，AllToAll = $(n-1)/n \times \text{vol}/\text{BW}$ ，AllGather = $(n-1)/n \times \text{vol}/\text{BW}$
- 利用率：计算 50%，HBM 带宽 80% (两平台相同)
- 内存检查： $\text{weight_per_rank} + \text{KV_cache_per_rank} \leq \text{HBM 容量}$
- Decode 近似：首尾 Decode 步之间的线性插值

附录 B：模型配置总结

参数	值
隐藏层大小	4,096
层数	43 (2 全注意力 + 21 C4A + 20 C128A)
Q 头数	64 (MQA: 1 个 KV 头)
头维度	512
Q LoRA 秩	1,024

参数	值
O 分组数	8, O LoRA 秩 = 1,024
索引头数	64, dim = 128, topK = 512
窗口大小	128 (SWA)
路由专家数	256, top-6
共享专家数	1
MoE 中间维度	2,048
HC 乘数	4
词表大小	129,280

附录 C：数据文件

所有原始数据位于 `report/data/`：

- `search_results_910C.json` / `search_results_H20.json` —— 各场景 top-20 配置
- `pd_ratio_analysis.json` —— P/D 比例计算
- `op_analysis.json` —— 算子级瓶颈分析
- `sp_comparison.json` —— SP/mHC-SP 对比
- `hardware_comparison.json` —— 跨平台对比
- `mhc_optimization_comparison.json` —— mHC 内核融合优化级别对比